How to Start a Project with an LLM — Step by Step (v1.4.0)

Oct 20, 2026 · @Herick

From idea to a version-controlled project folder (the scaffold), built with a coding agent such as Codex, OpenCode or Claude Code. The folder layout is a starting point, not fixed. Reference tables sit at the end so the steps run without interruption.

Before you start

- **You** decide and approve. The **planning chat** (any LLM) drafts documents. The **coding agent** builds in the real project folder.
- You need a terminal, Git and a coding agent. Node.js/npm only if you use `npx` skills.
- If a step does not apply, write **Not applicable** and go on.
- Never put passwords, tokens, confidential data or personal data in prompts, documents or Git. Use synthetic examples.
- **Tiny script?** Do Steps 1–3, put the few rules in `AGENTS.md`, skip Steps 4–12, and start at Step 13 by giving the agent `PROJECT_SPEC.md`. Anything bigger: do every step.

Start here: the master prompt

Open a new chat with your planning LLM (ChatGPT, Claude or another), paste this, and add your idea. It walks you through Steps 1–12; the steps below let you inspect or resume.

> Act as my interactive project-planning facilitator. I am a beginner. Follow this v1.4.0 standard. Ask one consequential question at a time and keep a running decision log. Interview me to challenge my assumptions. Explain the simplest suitable options with trade-offs. Distinguish requirements, recommendations, assumptions and unverified claims. Draft the specification and, where relevant, design, architecture and data contracts. Evaluate my required skills and MCP servers first, then optional ones; audit conflicts and security. Do not install tools or create the project during planning. Before handoff, give me a readiness report and, after my approval, a downloadable handoff package for Codex, OpenCode or Claude Code. Ask my approval for consequential decisions, security risks, substitutions and deviations. My idea: [DESCRIBE IDEA].

Planning is done when you approve the final specification and the handoff package.

Phase A — Decide what to build

Step 1 — Describe your idea

Write 2–5 sentences in everyday words: what it does, who uses it, the most important outcome.

Result: a short, informal description.

Step 2 — Write the specification

> Here is my idea: [PASTE IDEA]. Help me create a first-version project specification: purpose, intended users, 3–6 core features, what is out of scope, how we will know the first version works, a recommended technology stack, and a simple starter folder structure. Separate facts I supplied from your assumptions. Ask only about decisions that would materially change the plan. Save the approved specification as `PROJECT_SPEC.md`.

Start with the smallest useful version; add features later.

Result: approved `PROJECT_SPEC.md`.

Step 3 — Grill the requirements

> Interview me about the proposed project. Focus on missing requirements, conflicting assumptions, users, inputs and outputs, edge cases, success criteria and restrictions. Ask one question at a time. Keep a short decision log and update `PROJECT_SPEC.md` after I approve each answer. Stop when the project is clear enough to scaffold.

No skill is needed yet. Do not wait for every implementation detail.

Result: clearer requirements and a short list of decisions or open questions.

Step 4 — Draft the rule documents

Keep them short. Create only what applies:

- `ARCHITECTURE.md`: every project beyond a tiny script.
- `DESIGN.md`: if there is a user interface.
- `DATA_CONTRACTS.md`: if the project exchanges structured data, files, APIs, configuration or LLM output.

> Using `PROJECT_SPEC.md`, draft `ARCHITECTURE.md` (system parts, invariants, technology decisions). If there is a user interface, also draft `DESIGN.md` (design tokens, layout, components, loading/error behavior, responsiveness, accessibility). If the project has structured data inputs or outputs, draft `DATA_CONTRACTS.md` (producers, consumers, formats, semantics, validation rules). Match each document to the project's size. If something does not apply, say why. Flag contradictions among the documents before I approve them.

For `DESIGN.md`, require real values, not adjectives: hex color tokens, font stacks, spacing scale, corner radii, layout, components, keyboard accessibility, loading/error states, responsive rules.

Result: approved rule documents.

Step 5 — Define data contracts (if applicable)

Fix the exact shape of structured data before writing code that uses it. JSON Schema checks structure (types, required fields, enums, ranges). It does not check business rules such as formulas, uniqueness across records or matching totals; test those in code. For CSV, databases and other formats, document their native contract.

> From `PROJECT_SPEC.md`, `ARCHITECTURE.md` and `DATA_CONTRACTS.md`, list every data producer, consumer, input, output and transformation. For JSON, create schemas in `schemas/` using an explicitly stated JSON Schema draft (prefer 2020-12). Specify required fields, types, enums, nullability, units, dates, IDs, ranges, optional fields and error behavior. For other formats, document an equivalent contract. Generate synthetic valid, invalid and boundary examples, and list rules that need code-based tests. Do not invent field names or domain semantics; list open questions. Explain how contracts are validated and versioned.

Create only the folders you need: `schemas/inputs`, `schemas/outputs`, `schemas/config`, `schemas/examples/valid`, `schemas/examples/invalid`, `tests/contracts`. Breaking changes need review, migration or a coordinated release. Decide how versions are carried

(filenames, schema IDs or API versions).

Result: approved `DATA_CONTRACTS.md`, schemas, synthetic examples and a validation plan, or **Not applicable**.

Phase B — Choose skills and MCP servers

You only select and review here. Installation happens in Step 15, after the repository exists.

Step 6 — Review baseline skills

These are candidates, not an install-all list. Verify current names and sources first; upstream repositories change.

Skill	Use
`grill-with-docs` or `grill-me`	Challenge requirements. Pick one; `grill-with-docs` fits once project files exist.
`find-skills`	Discover skills for a new need. Discovery does not authorize installing.
`to-prd`, then `to-issues`	Turn the approved requirements into a PRD, then issues. Run `to-issues` only after PRD approval and issue-tracker setup.
`improve-codebase-architecture`	Review an existing codebase, not an empty scaffold.
`setup-matt-pocock-skills`	Configure that workflow in an initialized repository.

Inspect before installing, then install only approved skills (in Step 15, inside the project):

```
npx skills@latest add mattpocock/skills --list
npx skills@latest add vercel-labs/skills --list

npx skills@latest add mattpocock/skills --skill grill-with-docs
npx skills@latest add vercel-labs/skills --skill find-skills
npx skills@latest add mattpocock/skills --skill to-prd --skill to-issues
npx skills@latest add mattpocock/skills --skill improve-codebase-architecture
npx skills@latest add mattpocock/skills --skill setup-matt-pocock-skills
```

In place of `grill-with-docs`: `npx skills@latest add mattpocock/skills --skill grill-me`. Agents expose skills differently (slash command, named skill, other); ask your agent how to invoke each one. Sources: skills.sh/mattpocock/skills, skills.sh/vercel-labs/skills/find-skills.

Step 7 — Add your required skills

> The skills I require are: [LIST NAMES, REPOSITORIES OR LINKS]. Keep my choices separate from optional recommendations. Check compatibility with my coding agent. Recommend extra skills only if each serves a distinct purpose, and say when to use it. Do not install anything yet.

Global skills need not be copied into each project's `skills/` folder; record name, source, version, install scope and when to invoke in `SKILLS.md`.

Result: a skill manifest (required, recommended, accepted/rejected, reason, source, when to invoke), which becomes `SKILLS.md`.

Step 8 — Audit skill conflicts

> Examine the documentation and configuration of every selected skill. Find duplicates, contradictory instructions, overlapping responsibilities, dependency or version mismatches, permissions and competing configuration files. Write `SKILL_CONFLICT_REPORT.md` with columns: Skill A, Skill B, evidence, severity (Critical/Moderate/Minor), recommended resolution, status. Keep my required skills where technically and safely possible; ask me before substituting. Separate verified from potential conflicts. If none, write 'No skill conflicts detected' and say what you checked. Do not install skills with unresolved critical conflicts.

Result: an accepted `SKILL_CONFLICT_REPORT.md`. Compatible overlaps may stay; give each skill a distinct job.

Step 9 — List your MCP servers

An MCP server gives the agent tools or data (repositories, databases, services, documentation). It is optional; a simple offline script may need none.

> My required MCP servers are: [LIST, OR 'none']. For each, state what it connects to, which operations I expect, and whether my coding agent supports it. Suggest only complementary servers, with official sources and required permissions. Do not install or authorize anything yet.

Result: an approved `MCP_SERVERS.md`, or **No MCP servers needed**.

Step 10 — Audit MCP conflicts and permissions

> Audit every selected MCP server for duplicate tool names, competing writes, incompatible runtimes, transports, versions, ports, authentication, file and network access, and excessive permissions. Write `MCP_CONFLICT_REPORT.md` with evidence, severity, resolution and verification status. Do not enable unresolved critical conflicts. Never ask for secrets in chat or put credentials in repository files.

Result: an approved `MCP_CONFLICT_REPORT.md`, or **Not applicable**. Reviewing configuration does not prove a connection works.

Phase C — Hand off and scaffold

Step 11 — Run the planning readiness gate

> Audit my approved specification, `DESIGN.md` if applicable, `ARCHITECTURE.md`, `DATA_CONTRACTS.md` and schemas if applicable, selected skills and MCP servers, platform restrictions, security choices and test strategy. Find contradictions, missing decisions and unverified technical assumptions. Mark each finding BLOCKING or NON-BLOCKING and each check PASS, FAIL, SKIPPED or NOT VERIFIED, with the basis. Write `PLANNING_READINESS.md`. Keep grilling until blocking items are resolved, then ask for my explicit approval to hand off.

Result: approved `PLANNING_READINESS.md` and decision log. Never call an untested assumption verified.

Step 12 — Export and validate the handoff package

> Produce one downloadable archive, `PROJECT-HANDOFF-v0.1.0.zip`, with the approved plan: the files listed below that apply to this project, acceptance criteria, and a `MANIFEST.md` of included files and intentional omissions. Contents are specifications and proposed configuration, not a claim that the repository or tools exist. No secrets.

```
project-handoff/
  START-HERE.md          first action; how to give this package to the coding
agent
  HANDOFF.md             agent instructions and authorized actions
  PROJECT_SPEC.md        goals, scope, acceptance criteria
  DECISIONS.md           approvals, assumptions, open items
  PLANNING_READINESS.md  Step 11 report
  ARCHITECTURE.md        if needed
  DESIGN.md              if there is a UI
  DATA_CONTRACTS.md      if structured data is exchanged
  schemas/               if machine-readable contracts are needed
  SKILLS.md              requested/approved skills, sources, invocation
  MCP_SERVERS.md         approved servers and permissions, no credentials
  SECURITY.md            sensitivity, transmissions, permissions
  MANIFEST.md            included files and omissions
```

Check before Step 13:

1. Spec and architecture do not contradict each other.
2. Every mandatory user requirement is present.
3. Every required interface has an owner and a contract.
4. Unresolved critical conflicts are treated as blockers.
5. `SKILLS.md` and `MCP_SERVERS.md` separate requested, recommended, approved, installed and unverified entries.
6. No embedded secrets.
7. The agent's allowed actions and approval gates are explicit.
8. `START-HERE.md` names the first action.
9. Every file in `MANIFEST.md` exists, and no file is missing from it.

The package is a planning output, not proof that a scaffold exists. If your planning chat cannot make a ZIP, ask for the documents one by one; if a file cannot be generated, name the gap rather than invent a link.

Result: a handoff ZIP (or documents) that passed all nine checks.

Step 13 — Give the package to the coding agent

Open Codex, OpenCode, Claude Code or your chosen agent in a new project directory and supply the ZIP (or extracted folder):

> Read `START-HERE.md`, then `HANDOFF.md` and the files it references. Check this machine's actual software, permissions, internet restrictions and tools. Present your execution plan. Keep my approved decisions; ask before substitutions, architecture deviations, restricted network access or publishing. Create the complete initial scaffold with real starter files, not an empty template. Report anything you cannot complete.

Result: an execution plan you approve.

Step 14 — Create and version the scaffold

The agent creates working starter files and `README.md`, writes or updates `AGENTS.md` (or the agent's equivalent) linking to the design, architecture and contracts, checks `.gitignore`, runs `git init`, and makes the first clean commit. Secrets stay in ignored local configuration. No force-push, overwriting or publishing without approval.

Setup entry points depend on size:

- **Tiny script:** one documented run command in `README.md`.
- **Anything larger:** `scripts/setup.sh` (Bash), `scripts/setup.ps1` (PowerShell), `scripts/setup.cmd` (Command Prompt), with equivalent documented tasks. Do not assume the same package managers or admin rights. Test each where possible; mark the rest NOT VERIFIED.

Result: a real repository, the setup entry points your size needs, and a traceable first commit.

Step 15 — Install approved tools and set up agent context

The agent installs only approved skills and MCP servers, in their verified scope (global or project-local), and checks names, versions, capabilities and connections. It records results and approved substitutions in `SKILLS.md` and `MCP_SERVERS.md`, runs `/init` only where supported (otherwise the tool's documented equivalent), and confirms the agent instructions reference `DESIGN.md`, `ARCHITECTURE.md`, `DATA_CONTRACTS.md` and the skills. No credentials in Git.

Result: agent context and tools configured, or marked unavailable or unverified.

Step 16 — Verify the scaffold

> Run every check this environment supports: starter program, dependencies, configuration, schema examples, tests, skill invocation, approved MCP connections and the platform setup scripts. Report each as PASS, FAIL, SKIPPED or NOT VERIFIED with the command or evidence. Do not infer that untested scripts work. Write `SCAFFOLD_READINESS.md` with the Git commit ID and copy/paste instructions to run, test and resume. Commit validated changes; do not publish unless I authorize it.

Correct or disclose every FAIL. NOT VERIFIED is not PASS.

Result: a verified baseline, an evidence-backed `SCAFFOLD_READINESS.md`, a list of unfinished tasks, and your next action.

Phase D — Build and keep the project healthy

Step 17 — Build the first small feature

The scaffold is not the application. Build in small, tested changes. With Matt Pocock's workflow, refine the PRD with `to-prd` and run `to-issues` after PRD approval and tracker setup; otherwise a task checklist is enough.

> Read the repository's project rules. Propose the smallest useful feature to build next, with acceptance criteria and a way to test it. Name the files and data contracts it affects. Implement one reviewable change at a time, run the right tests, summarize what changed, and commit only after I approve or explicitly authorize it.

Result: an incremental, testable development cycle.

Step 17A — Parallelize issues with sub-agents (optional)

Only after PRD approval and an issue list with acceptance criteria, dependencies, owners and affected interfaces. Not every issue needs its own agent. Skip this step if parallel work adds more overhead than value.

> Inspect the approved `to-issues` output. Build a dependency graph and find independent issues with non-overlapping write sets and stable interfaces. Propose a parallel plan: one orchestrator, a worktree or branch per worker, issue ownership, integration order, test requirements and a serial fallback. Explain race, shared-file and integration costs. Ask my approval before dispatching sub-agents or enabling paid services. Never run dependent work in parallel without an agreed interface contract.

The orchestrator owns the issue graph and final integration, re-runs integration and contract tests, and documents results. Workers do not edit the same shared files without coordination. If the agent cannot launch sub-agents, run the same plan serially.

Result: an approved parallel plan with integrated, verified issues, or a documented serial plan.

Step 18 — Let the project evolve; keep documents in step

Folders, modules and dependencies will change. That is expected. The setup scripts record the initial setup; they are not an order to restore the original layout. Keep `README.md`, `ARCHITECTURE.md`, `DESIGN.md` and `DATA_CONTRACTS.md` matching the real software.

Before accepting a change, ask:

> Does this change affect the architecture, UI rules, data producers or consumers, schemas, skills, MCP servers, permissions or build commands? If so, update the documents and tests in the same change. Check schema compatibility and migrations before changing data interfaces. Re-run conflict audits if skills or MCP servers changed. Show me the verification results and unresolved risks.

When the structure changes materially, update `AGENTS.md`; review any automated `/init` output, because it can overwrite custom guidance. Use `improve-codebase-architecture` only once meaningful code exists. Commit useful checkpoints.

Result: a growing project that you and future agent sessions can still understand.

Reference

Look things up here; none of it interrupts the steps.

A. Which file does what

File or folder	Audience	Role	Create or update when
`README.md`	People	What the project does; how to start it	Scaffold; user-facing changes
`PROJECT_SPEC.md`	You, agent	Scope, goals, acceptance criteria	Planning; changed requirements
`DECISIONS.md`	You, agent	Approvals, assumptions, open items	Every consequential decision
`AGENTS.md`	Coding agent	Commands, conventions, safety rules, where other rules live	After the scaffold exists; as workflows change
`ARCHITECTURE.md`	Agent, maintainers	Non-obvious invariants and rationale	Architecture changes
`DESIGN.md`	Agent, UI contributors	Concrete UI tokens and interaction rules	UI exists or changes
`DATA_CONTRACTS.md`, `schemas/`	Producers, consumers, validators	Meaning, shape and compatibility of exchanged data	Data interfaces change
`SKILLS.md`, `skills/`	You, agent	Skill inventory; optional project-local skills (each with `SKILL.md`)	Skill selection or versions change

File or folder	Audience	Role	Create or update when
`MCP_SERVERS.md`	You, agent	Servers, permissions, setup; never secrets	MCP choices change
`SECURITY.md`	You, agent	Sensitivity, external transmissions, permissions	Security-relevant choices change
`START-HERE.md`, `HANDOFF.md`	You, coding agent	Handoff and first action (planning output)	Step 12
`PLANNING_READINESS.md`	You, planning chat	Readiness before handoff	Step 11
`SCAFFOLD_READINESS.md`	You, coding agent	Check results after scaffolding	Step 16 and later
`scripts/`	You, agent	Reproducible initial setup	Scaffold; intentional change

`README.md` explains the project to people; `AGENTS.md` tells a coding agent how to work in this repository; a `SKILL.md` describes a reusable procedure for many repositories. Some agents use their own instruction file; verify what yours loads.

B. Rules

Topic	Rule
Approval	The agent decides routine details. Significant changes, conflicting requirements, security-sensitive actions and changes to your approved choices need your approval.
Git	Commits, a branching policy, version identifiers, relevant tests. Approval before destructive Git operations or publishing.

Topic	Rule
Security	Assess sensitivity, external transmissions, permissions and credential storage. Never commit secrets or send restricted data without permission.
Verification	Report PASS / FAIL / SKIPPED / NOT VERIFIED with evidence. Never claim a check ran when it did not.
Evolution	The tree can change. Review affected documents, contracts and tests on meaningful changes.

Agent may proceed: ordinary implementation details within approved scope, routine tests, fixing reversible setup errors, updating documentation, committing when authorization covers it.

Ask you first: changing core requirements or architecture; replacing a skill or MCP server you chose; wider data access or external transmission; paid services; deleting data; rewriting Git history; publishing or deploying; dispatching parallel workers. Record approvals in `DECISIONS.md`.

No answer from you: stop only the affected operation, continue independent safe work, and mark the blocked item. Silence is not approval.

C. Three completion checkpoints

Milestone	Confirmed by	Evidence
Planning complete	You, with the planning chat	Approved `PROJECT_SPEC.md`, relevant drafts, risks, decisions, handoff package
Scaffolding complete	Coding agent, with your review	Repository, real files, Git baseline, `SCAFFOLD_READINESS.md`
Development ready	You and the coding agent	Usable agent instructions, applicable checks verified, blockers named, first task named

Never substitute one for another: a handoff ZIP is not a scaffold, and a commit is not proof that tests passed.

D. Agent compatibility: verify, do not assume

Capability	Codex	OpenCode	Claude Code	Action
Repository instructions	Commonly `AGENTS.md`	Varies by configuration	Commonly `CLAUDE.md`, not `AGENTS.md`; a common bridge is a `CLAUDE.md` that imports `AGENTS.md` (check the current syntax)	Load or import the project instructions
Initialization	Depends on version	`/init` where supported	`/init` where supported	Check the tool's help
Skills and commands	Supported environments	Version/plugin dependent	Supported environments	Confirm path, discovery and invocation
MCP servers	Supported configurations	Supported configurations	Supported configurations	Check transport, permissions, a harmless test call
Sub-agents	Configuration-dependent	Configuration-dependent	Configuration-dependent	Verify support and budget; otherwise run serially

These are orientation notes, not guarantees. The agent must verify its installed version and record actual results before claiming PASS.

E. When something fails

1. Stop repeating the failing command. Paste the exact error and command into the agent chat, without secrets.

2. Ask: "Explain the failure in plain language. Is it a missing prerequisite, wrong terminal, restricted permission, version mismatch, credential problem, network issue, dependency conflict or genuine code defect?"
3. Ask for the smallest reversible fix, what it changes, and how to confirm it. Get approval first if it touches security, your choices or published history.
4. Re-run the check and record PASS / FAIL / SKIPPED / NOT VERIFIED in `SCAFFOLD_READINESS.md` with command, platform and output.
5. If the environment cannot support the check, mark NOT VERIFIED, document a manual test, and continue only if nothing depends on it.
6. If setup is damaged, go back to the last good commit without discarding uncommitted work; check `git status` and get approval before destructive operations.

Skill not found: confirm its exact name and install scope, then reload the agent. MCP connection fails: check permissions and credentials without exposing secrets. Git push fails: keep local commits and diagnose the remote or authorization. ZIP or PDF download fails: use the committed Markdown.

F. Dry run: the acceptance test for this standard

Run this guide once on a non-sensitive toy project (for example, a CLI that validates a small synthetic JSON file) and record date, environment, commands and outcomes.

- [ ] The planning chat produced an approved handoff package and the agent found `START-HERE.md`.
- [ ] The agent built real files and committed the baseline.
- [ ] Above tiny-script size: Bash, PowerShell and CMD entry points exist and are equivalent; untested ones are NOT VERIFIED.
- [ ] Starter run, skill discovery, MCP checks and tests have recorded results.
- [ ] You can name the next action and recover from an induced, reversible failure.
- [ ] The English and Spanish editions have the same steps and release number.

Dry-run status for v1.4.0: **NOT VERIFIED.** No run has been recorded for this release or for 1.3.1.

G. Quick completion checklist

- [ ] `PROJECT_SPEC.md` states goals and boundaries.

- [ ] `ARCHITECTURE.md` states real rules (for a tiny script, `AGENTS.md` does); `DESIGN.md` exists if there is a UI.
- [ ] Data producers and consumers are identified; schemas and synthetic tests exist where needed.
- [ ] Required skills and MCP servers are separate from optional ones; critical conflicts are resolved before installation.
- [ ] The scaffold has real files and a runnable starter.
- [ ] Git has a baseline commit and no secrets.
- [ ] The agent can read the project rules and invoke approved tools.
- [ ] Baseline tests pass, or failures are documented.
- [ ] You understand the initial tree is a starting point.

The whole process: idea → specification → clarify → design, architecture, data contracts → choose and audit skills and MCP → handoff package → scaffold → Git baseline → configure tools → verify → build small features → keep documents, schemas, tests and Git current.

H. Versions of this standard

This guide is a living standard, separate from the version of any project you start with it. MAJOR.MINOR.PATCH: MAJOR for incompatible workflow changes, MINOR for new compatible guidance, PATCH for editorial fixes. Each release updates the title, filename, metadata and this table, and keeps the old version in Git or an archive. Suggested commit: `docs(standard): release v1.4.0`; tag: `project-standard-v1.4.0`. English and Spanish editions must share steps and release number.

Version	Date	Changes
1.4.0	2026-10-20	Steps now run uninterrupted; reference material moved to the end; wording cut. One consistent handoff file list; Step 12 merged with its validation (check 9 added); `ARCHITECTURE.md` rule aligned; Step 6 install reference corrected to Step 15; planning chat made LLM-agnostic; setup scripts tiered by size; tiny-script path added; `PLANNING_READINESS.md` vs `SCAFFOLD_READINESS.md` separated; Claude Code `CLAUDE.md` note; dry-run status stated as NOT VERIFIED; Spanish edition added. Unchanged by decision: unpinned `npx skills@latest`.
1.3.1	2026-10-18	Completion checkpoints, approval gates, validated handoff package, agent compatibility notes, troubleshooting, dry-run protocol, cross-platform wording, corrected revision history.
1.3.0	2026-10-18	18-question grilling decisions, interactive master prompt, structured agent handoff, cross-platform setup, evidence-based readiness, optional sub-agents.
1.2.0	2026-10-18	Merged Project Bootstrap Kit concepts: file responsibility matrix, global vs local skills, design tokens, minimum configurations, agent-file portability, scaffold regeneration safeguards.
1.1.0	2026-10-18	Literal beginner workflow; `PROJECT_SPEC.md`, conditional `DESIGN.md` and `DATA_CONTRACTS.md`, JSON Schema, tool install timing, scaffold checks, agent initialization.
1.0.0	2026-10-18	First baseline: skills, MCP selection and installation, conflict audits, scaffolding, agent context, Git.